**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**Дисциплина:** Инженерия данных

# Отчет
# Домашняя работа №4
# “Apache ZooKeeper”

Выполнила:
Клемяционок Полина

# Этап 1. Установка и подготовка

Выбрала вариант с Docker: подняла три сервера из официального образа zookeeper:3.9.6 через compose.yaml. Java отдельно ставить не пришлось, в образе уже есть Java 17 (в выводе zkCli видно java.version=17.0.21). Координатор написан на Python

```
pklem@DESKTOP-J689SSF MINGW64 C:/Users/pklem/Documents/hw4_zookeeper/repo (master)
$ docker compose ps
zk1       zookeeper:3.9.6   "/docker-entrypoint.…"   zk1       9 minutes ago   Up 4 minutes   0.0.0.0:2181->2181/tcp, [::]:2181->2181/tcp
zk2       zookeeper:3.9.6   "/docker-entrypoint.…"   zk2       9 minutes ago   Up 4 minutes   0.0.0.0:2182->2182/tcp, [::]:2182->2182/tcp
zk3       zookeeper:3.9.6   "/docker-entrypoint.…"   zk3       9 minutes ago   Up 6 minutes   0.0.0.0:2183->2183/tcp, [::]:2183->2183/tcp
```

# Этап 2. Ансамбль ZooKeeper

Для каждого сервера написала свой conf/zoo.cfg. Строки server.1–server.3 во всех трёх одинаковые, а clientPort (2181, 2182, 2183) и dataDir (/data/zk1, /data/zk2, /data/zk3) у каждого свои, чтобы серверы не перезаписали данные друг друга. В dataDir лежит файл myid с номером сервера (1, 2 и 3), по нему сервер понимает, какая строка server.N про него

```
tickTime=2000
initLimit=10
syncLimit=5
dataDir=/data/zk1
clientPort=2181
4lw.commands.whitelist=*
admin.enableServer=false
server.1=zk1:2888:3888;2181
server.2=zk2:2888:3888;2182
server.3=zk3:2888:3888;2183
```

В контейнере сервер запускается штатной командой zkServer.sh start-foreground, роли смотрела через zkServer.sh status. Из-за того что сервер там главный процесс, команда zkServer.sh stop его не видит, поэтому останавливала серверы через docker stop и запускала через docker start

Роли: zk2 оказался лидером, zk1 и zk3 последователями

```
$ docker exec zk1 zkServer.sh status
Mode: follower

$ docker exec zk2 zkServer.sh status
Mode: leader

$ docker exec zk3 zkServer.sh status
Mode: follower
```

Остановила один сервер (сразу лидера zk2, чтобы было интереснее). Ансамбль продолжил работать: лидером стал zk3, узел /test_one_down создался и читается, так как 2 сервера из 3 это кворум

```
$ docker exec zk1 zkServer.sh status
Mode: follower

$ docker exec zk2 zkServer.sh status
Error response from daemon: container 253b2ed788b1f016310fd2101e9ef670fa75f9c8c9ca22f670413198c700e0a3 is not running

$ docker exec zk3 zkServer.sh status
Mode: leader

$ docker exec zk1 zkCli.sh -server localhost:2181 create /test_one_down hello
Created /test_one_down

$ docker exec zk1 zkCli.sh -server localhost:2181 get /test_one_down
hello
```

Остановила второй сервер zk1. Оставшийся zk3 запущен, но запросы не обслуживает: кворума нет, ансамбль не отвечает

```
$ docker exec zk1 zkServer.sh status
Error response from daemon: container e472e3af6f110e358e00f2efbfbd0ced780cbd9695e413691fff856c5572307f is not running

$ docker exec zk2 zkServer.sh status
Error response from daemon: container 253b2ed788b1f016310fd2101e9ef670fa75f9c8c9ca22f670413198c700e0a3 is not running

$ docker exec zk3 zkServer.sh status
Error contacting service. It is probably not running.

$ timeout 25 docker exec zk3 zkCli.sh -server localhost:2183 ls / | grep KeeperErrorCode
KeeperErrorCode = ConnectionLoss for /

$ docker exec zk3 bash -c 'echo srvr | nc -w 2 localhost 2183'
This ZooKeeper instance is not currently serving requests
```

Запустила оба сервера снова: опять один лидер и два последователя, узел /test_one_down на месте

```
$ docker start zk2 zk1
zk2
zk1

$ docker exec zk1 zkServer.sh status
Mode: follower

$ docker exec zk2 zkServer.sh status
Mode: follower

$ docker exec zk3 zkServer.sh status
Mode: leader

$ docker exec zk1 zkCli.sh -server localhost:2181 get /test_one_down
hello
```

Эфемерный узел: во втором окне zkCli создала create -e /ephemeral_demo. Из первого окна узел виден, и у него есть ephemeralOwner (id сессии второго окна). После того как закрыла второе окно, узел исчез, потому что он живёт только пока жива сессия,

которая его создала

```
[zk: localhost:2182(CONNECTED) 0] create -e /ephemeral_demo 'session-owned node'
Created /ephemeral_demo

$ docker exec zk1 zkCli.sh -server localhost:2181 ls /
[ephemeral_demo, test_one_down, zookeeper]

$ docker exec zk1 zkCli.sh -server localhost:2181 stat /ephemeral_demo | grep ephemeralOwner
ephemeralOwner = 0x2000002e49c0000

[zk: localhost:2182(CONNECTED) 1] quit

$ docker exec zk1 zkCli.sh -server localhost:2181 ls /
[test_one_down, zookeeper]

$ docker exec zk1 zkCli.sh -server localhost:2181 get /ephemeral_demo
Node does not exist: /ephemeral_demo
```

# Этап 3. Приостановка лидера без защиты

В дз №3 у меня был скрипт mapreduce.py, который считает навыки, пары и тройки навыков по вакансиям Python-разработчиков. Эту же логику без изменений вынесла в skills.py и сделала координатор coordinator.py, который работает через ZooKeeper:

- выбор лидера через рецепт Election из kazoo (эфемерные последовательные узлы), тайм-аут сессии 10 секунд
- эпоха: каждый новый лидер обновляет узел epoch, и его версия становится номером эпохи лидера
- чекпоинт после каждых 50 вакансий: счётчики сохраняются в файл, а в узел checkpoint записывается, сколько уже обработано
- результат публикуется в узел-указатель result. Он создается пустым (версия 0), и каждая запись увеличивает версию на 1

Запустила две копии координатора (A и B) через run_failover.py. Когда лидер A обработал 400 из 1076 вакансий, приостановила его процесс (аналог kill -STOP) на 40 секунд, а потом возобновила.

```
ДЗ №3
Уникальных навыков: 868
Уникальных пар: 9779
Уникальных троек: 48407
КОНТРОЛЬ: вакансий=1076 навыков=868 пар=9779 троек=48407 сумма_пар=20843 топ_пара=postgresql+{python:226
```

```
10:31:50.367 [runner] RUN=stage3_nofencing FENCING=0: запускаю coordinator-A
10:31:50.907 [runner] coordinator-A - лидер; запускаю запасной coordinator-B
10:32:01.303 [runner] ПРИОСТАНОВИЛ лидера coordinator-A (pid 33192) посреди расчёта: {"offset": 400, "file": "checkpoints/stage3_nofencing.json", "by": "coordinator-A"
, "epoch": 1}
10:32:12.630 [runner] ЛИДЕР СМЕНИЛСЯ на coordinator-B через 11.33 с после приостановки A
10:32:33.218 [runner] coordinator-B завершил работу через 31.9 с после приостановки A
10:32:41.301 [runner] ВОЗОБНОВИЛ coordinator-A через 40.0 с паузы
10:32:58.746 [runner] coordinator-A завершил работу
10:32:58.748 [runner] ИТОГ: указатель /hw4/stage3_nofencing/result версия=2 значение={"writer": "coordinator-A", "epoch": 1, "file": "results/stage3_nofencing_coordina
tor-A.json"}
10:32:58.750 [runner] ИТОГ: эпоха (версия /hw4/stage3_nofencing/epoch) = 2
10:32:58.750 [runner] ИТОГ: время смены лидера = 11.33 с (тайм-аут сессии 10 с)
```

```
10:31:50.860 [coordinator-A] СТАЛ ЛИДЕРОМ
10:31:50.890 [coordinator-A] чекпоинта нет, начинаю с нуля: всего вакансий 1076
10:32:58.704 [coordinator-A] КОНТРОЛЬ: вакансий=1076 навыков=868 пар=9779 троек=48407 сумма_пар=20843 топ_пара=postgresql+{python:226
10:32:58.712 [coordinator-A] записал результат: /hw4/stage3_nofencing/result -> results/stage3_nofencing_coordinator-A.json, версия указателя 2, эпоха 1
10:32:12.578 [coordinator-B] СТАЛ ЛИДЕРОМ
10:32:12.709 [coordinator-B] продолжаю с чекпоинта: обработано 400/1076 (чекпоинт записал coordinator-A, эпоха 1)
10:32:33.150 [coordinator-B] КОНТРОЛЬ: вакансий=1076 навыков=868 пар=9779 троек=48407 сумма_пар=20843 топ_пара=postgresql+{python:226
10:32:33.159 [coordinator-B] записал результат: /hw4/stage3_nofencing/result -> results/stage3_nofencing_coordinator-B.json, версия указателя 1, эпоха 2
```

Лидер сменился через 11,33 секунды после приостановки A. Это тайм-аут сессии 10 секунд плюс время, за которое ZooKeeper замечает истекшую сессию и удаляет эфемерный узел. Запасной B не начал сначала, а продолжил с чекпоинта 400/1076. Контрольные числа совпали с дз №3: 868 навыков, 9 779 пар и 48 407 троек.

Но A после разморозки не знает, что он уже не лидер: он спокойно досчитал и записал результат второй раз. В журналах две строки «записал результат», у указателя версия 2, и теперь в нём результат старого лидера с эпохой 1.

# Этап 4. Приостановка лидера с защитой по эпохе

Повторила то же самое с FENCING=1. Теперь результат записывается одной транзакцией из двух шагов: сначала проверка, что версия узла epoch всё ещё равна эпохе лидера (check), потом сама запись (set_data). Если пока лидер спал кто-то другой стал лидером, эпоха уже другая и вся транзакция отклоняется.

```
10:33:09.528 [coordinator-A] СТАЛ ЛИДЕРОМ
10:33:09.554 [coordinator-A] чекпоинта нет, начинаю с нуля: всего вакансий 1076
10:34:19.031 [coordinator-A] КОНТРОЛЬ: вакансий=1076 навыков=868 пар=9779 троек=48407 сумма_пар=20843 топ_пара=postgresql+{python:226
10:34:19.048 [coordinator-A] ОТКАЗ ЗАПИСИ: BadVersionError - моя эпоха 1, текущая эпоха 2; я больше не лидер, результат НЕ записан
10:33:30.570 [coordinator-B] СТАЛ ЛИДЕРОМ
10:33:30.707 [coordinator-B] продолжаю с чекпоинта: обработано 400/1076 (чекпоинт записал coordinator-A, эпоха 1)
10:33:50.728 [coordinator-B] КОНТРОЛЬ: вакансий=1076 навыков=868 пар=9779 троек=48407 сумма_пар=20843 топ_пара=postgresql+{python:226
10:33:50.752 [coordinator-B] записал результат: /hw4/stage4_fencing/result -> results/stage4_fencing_coordinator-B.json, версия указателя 1, эпоха 2
```

```
10:33:09.094 [runner] RUN=stage4_fencing FENCING=1: запускаю coordinator-A
10:33:09.632 [runner] coordinator-A - лидер; запускаю запасной coordinator-B
10:33:19.989 [runner] ПРИОСТАНОВИЛ лидера coordinator-A (pid 24884) посреди расчёта: {"offset": 400, "file": "checkpoints/stage4_fencing.json", "by": "coordinator-A",
"epoch": 1}
10:33:30.616 [runner] ЛИДЕР СМЕНИЛСЯ на coordinator-B через 10.63 с после приостановки A
10:33:50.790 [runner] coordinator-B завершил работу через 30.8 с после приостановки A
10:33:59.987 [runner] ВОЗОБНОВИЛ coordinator-A через 40.0 с паузы
10:34:19.105 [runner] coordinator-A завершил работу
10:34:19.108 [runner] ИТОГ: указатель /hw4/stage4_fencing/result версия=1 значение={"writer": "coordinator-B", "epoch": 2, "file": "results/stage4_fencing_coordinator-
B.json"}
10:34:19.109 [runner] ИТОГ: эпоха (версия /hw4/stage4_fencing/epoch) = 2
10:34:19.110 [runner] ИТОГ: время смены лидера = 10.63 с (тайм-аут сессии 10 с)
```

Лидер сменился через 10,63 секунды, B снова продолжил с чекпоинта, и контрольные числа снова совпали с дз №3. А вот запись возобновившего работу A теперь отклонена с BadVersionError. Результат остался один: указатель версии 1 от запасного координатора B.

| Запуск | смена лидера | совпало с ДЗ №3 | записей результата | версия указателя | чей результат |
|---|---|---|---|---|---|
| без защиты | 11,33 с | да | 2 | 2 | A, старая эпоха 1 |

| | | | | | |
|---|---|---|---|---|---|
| с защитой | 10,63 с | да | 1 (+ BadVersionError у A) | 1 | B, эпоха 2 |

# Этап 5. Оформление результатов

В репозитории лежат compose.yaml и конфигурации серверов (zk1–zk3/conf/zoo.cfg, myid), журналы всех запусков координатора в папке logs и выводы всех команд текстом в папке outputs.

**Почему серверов три, а не два и не четыре?**

ZooKeeper работает, пока живо большинство серверов (кворум). При двух серверах кворум это 2, то есть падение любого из них останавливает всё, и по итогу это даже хуже одного сервера. При трёх кворум тоже 2, и один сервер может упасть, это я и проверила на этапе 2. А при четырёх кворум уже 3, и упасть может тоже только один, так что четвёртый сервер ничего не даёт, только запись становится медленнее, ведь нужно ждать подтверждения от лишнего узла. Поэтому берут нечётное число: 3 сервера выдерживают отказ одного, 5 серверов отказ двух. И ещё правило большинства не даёт ансамблю развалиться на две половины с двумя лидерами, потому что две группы не могут одновременно быть большинством.

**Почему защиту по эпохе проверяет получатель записи, а не ZooKeeper?**

ZooKeeper может только сказать, кто лидер сейчас, но остановить старого лидера он не может. Это и видно на этапе 3: A был заморожен, не знал, что его сессия истекла, и после разморозки просто продолжил работать. Даже если лидер проверит «я ещё лидер?» перед записью, между проверкой и записью он может зависнуть (сборка мусора, пауза виртуалки, сеть), и к моменту записи эта проверка уже неактуальна. Надежно проверить можно только в самый момент приёма, и сделать это может только тот, кто запись принимает: он сравнивает эпоху в запросе с самой новой и откидывает старую. ZooKeeper тут только выдаёт номера эпох. У меня получатель это узел-указатель, и проверка встроена в саму запись. А если бы результат писался, например, в S3 или базу, проверять эпоху пришлось бы им.

[гитхаб]